24-786 Project Final Report
Using Graphs of Convex Sets for Point-Robot and Manipulation Planning

Trevor Stack (tstack)

April 25, 2026

***Abstract*— This project explores how to generate high-quality, collision-free paths for point-robots and manipulators by optimizing over a graph of convex regions.**
All code for this project can be found here:
https://github.com/Trevor-Stack/GCS_Planning_Project.git
The final presentation slides can be found here:
https://docs.google.com/presentation/d/1eEx23Pf4ArSnRoN660epyI2_gxAgnMcG-5Q4rHJfaCc/edit?usp=sharing

I. Introduction

Motion planning in robotics is the process of computing a collision-free path from a starting configuration to a goal configuration that satisfies the robotic platform's kinematic and dynamic constraints. The structure and complexity of the planning problem depend on the robotic platform being deployed. For example, a holonomic ground robot traversing over a flat factory floor can plan directly in the 2-dimensional work space (x, y) with no constraints on the direction of its instantaneous velocity. A car-like robot with non-holonomic constraints cannot move sideways. As a result, heading bust be included into the state space (x, y, θ) and edges between nodes on the planning graph must satisfy the platform's kinematic constraints. A 7 degree-of-freedom robot arm plans in a 7-dimensional configuration space. Due to nonlinear forward kinematics, obstacles and self-collisions map to complex, non-linear regions in configuration space, making exploring free-space much more challenging.

This project focuses on using Graphs of Convex Sets (GCS) [1] to generate trajectories for a holonomic robot and a robot arm. GCS uses convex optimization to generate high-quality, collision-free trajectories around obstacles. GCS formulates the planning problem by discretizing free-space into convex regions and creating a graph where each node is a convex region and each edge is a transition between regions. By solving a Mixed Integer Program (MIP) over the graph, it finds a combination of trajectories between and through convex regions that creates a low-cost, collision-free path to the goal.

II. Related Work

Search based planners, like A* [2], and sampling-based planners, like RRT* [3], have unique advantages and limitations. A* discretizes free-space into cells/voxels and expands nodes until the goal is reached. A* is complete and optimal, meaning A* will eventually return the best solution over the discretized free-space if a solution exists. However, the quality of the solution is highly dependent on the resolution of the discretization. Additionally, A* becomes computationally intractable for high dimensional problems.

RRT* incrementally builds a tree by sampling a configuration in the continuous state space and connecting an existing node to the new sample. This method is probabilistically complete and asymptotically optimal, meaning as the number of samples approaches infinity, the probability of finding an existing solution approaches 1 and the solution quality will approach the optimum. However, RRT* may return highly sub-optimal solutions and may struggle with narrow passageways.

Unlike A* and RRT*, GCS explicitly handles convex constraints, enabling it to generate smooth, kinematically feasible trajectories without post-processing or having to sample the configuration space extensively.

CHOMP [4] is a widely used gradient-based trajectory optimization method that produces smooth paths in continuous space. However, CHOMP is a local optimizer that iteratively improves an initial trajectory guess. Additionally, collision avoidance is incorporated into the objective function and not treated as a hard constraint. Therefore, if the initial guess is poor, CHOMP may get stuck in a local minimum and return a low-quality or colliding trajectory.

GCS overcomes the limitations of CHOMP by decomposing the collision-free configuration space into convex regions and constraining the trajectory to lie within the convex regions.

III. Motivation

When applying GCS to a robotics problem, it is important to consider how to decompose the free configuration space into convex regions, how to tune the objective function and constraint parameters, and whether to use convex relaxation.

The convex decomposition of configuration space becomes a complex problem when the configuration space is high-dimensional and non-convex with nonlinear boundaries. If the the convex decomposition does not cover enough of the actual configuration space, GCS may not be able to find a feasible trajectory even though a solution exists.

Careful tuning of trajectory constraints (e.g., continuity, velocity bounds, bezier curves) and cost function weights (energy, path length, or time) is required to produce a path with the desired behavior.

Convex relaxation can greatly improve computational efficiency. However, it may return a suboptimal solution if the relaxation is not tight.

To understand these trade-offs, this project implements GCS for a point robot and a 3-joint planar robot arm and evaluates:

- Trajectory quality
- Impact of parameter tuning
- Tightness of the convex relaxation

IV. Problem Setup

A. Point Robot on a 2D Plane

The first problem is to plan a trajectory for a holonomic point robot moving on a 2D plane with polygon obstacles. The environment with the start and goal positions are shown in Figure 1.

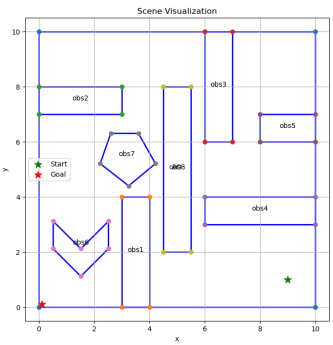
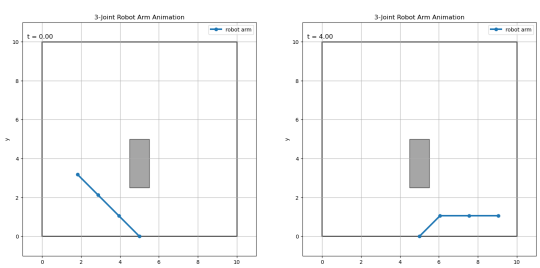


Fig. 1: Point Robot - Map 1

B. Robot Arm with 3 Joints

The second problem is to plan a trajectory for a planar robot arm with 3 joints. In the code repository (link to repository), I tested GCS on several different environments. This report will discuss the results for map 3. The start and end configurations map 3 are shown in Figure 2. The trajectory must avoid collisions with obstacles as well as self-collisions.



(a) Start (b) End

Fig. 2: Robot Arm - Map 3

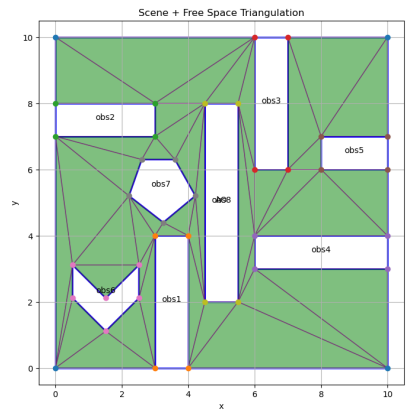
V. Method

The process of implementing GCS involves 3 steps:

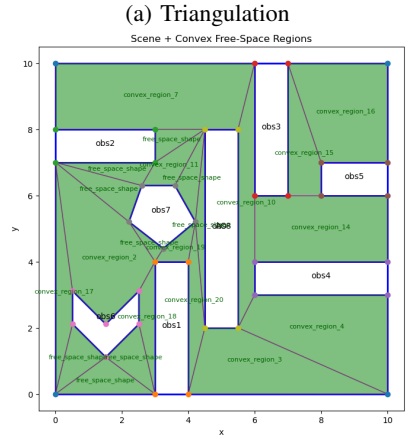
1) Decomposing configuration space into convex regions
2) Creating the graph of convex sets
3) Solving the MIP over the graph with or without convex relaxation.

A. Convex Decomposition of Free Configuration Space

1) Decomposing 2D Free Space Around Polygon Obstacles: As discussed in Section I, a holonomic ground robot can plan directly in its 2D workspace (x, y). To decompose the workspace into convex regions, I used Constrained Delauney Triangulation (CDT) from the CGAL library [5] to triangulate the 2D workspace. Then I merged triangles into larger convex regions to help minimize the number of decision variables in the GCS optimization problem. The convex decomposition of map 1 is shown in Figure 3.



(a) Triangulation



(b) Convex Regions

Fig. 3: Free-space decomposition: triangulation and merging into convex regions

2) Decomposing 3D Configuration Space Around Nonlinear Obstacles: For a robot arm with 3 revolute joints, the configuration space is 3-dimensional. The robot's forward kinematics is dictated by Equations 2 and 4:

$$x = l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) \tag{1}$$
$$+ l_3 \cos(q_1 + q_2 + q_3), \tag{2}$$
$$y = l_1 \sin(q_1) + l_2 \sin(q_1 + q_2) \tag{3}$$
$$+ l_3 \sin(q_1 + q_2 + q_3). \tag{4}$$

where:

- l_i is the length of link i
- q_i is the angle of joint i measured relative to the previous link
- (x, y) is the position of the end effector in the plane

Because the forward kinematics is nonlinear, convex obstacles in task space map to complex, non-convex obstacles in configuration space. Figure 4 from [6] illustrates this for a 2 degree of freedom arm.

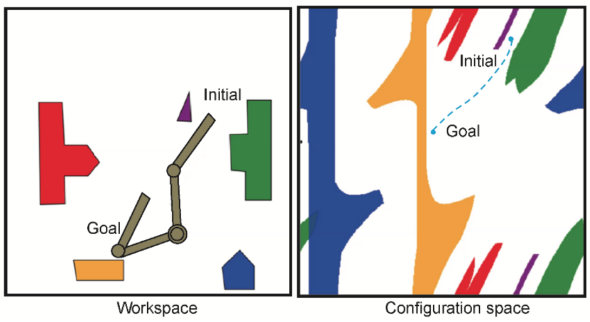


Fig. 4: Mapping of Workspace Obstacles to Configuration Space

Therefore, for the 3 degree of freedom robot arm, using CDT to triangulate the free configuration space is not feasible.

To decompose the non-convex configuration space into convex regions, I used Configuration-space Iterative Regional Inflation by Semidefinite programming (C-IRIS) [7]. C-IRIS first samples valid configurations to form a small initial convex region. Then sequentially solves 3 convex optimization problems:

1) Proving the current convex region is not in collision
2) Measuring the size of an ellipse that fits inside the convex region
3) Finding hyperplanes between the current region and the nearest obstacles and expanding the convex region

Equation 5 is the 3rd optimization problem, simplified from the C-IRIS paper [7]. Its objective is to expand the sides of a polytope (the convex region) as far as possible without intersecting with a collision region. δ_i is the outward expansion of face i, s is the robot configuration, m is the number of sides of the polytope, and $A(s)$ and $B(s)$ represent convex collision geometries. $A(s)$ or $B(s)$ can be the collision geometry of a robot arm link or a static obstacle in the environment. The polytope expansion is depicted by Figure 5 from [7].

$$\max_{\delta} \quad \prod_{i=1}^{m} (\delta_i + \epsilon_0) \tag{5a}$$

$$\text{s.t.} \quad \delta_i \geq 0 \quad \forall i \in [m] \tag{5b}$$

$$A(s) \text{ and } B(s) \text{ are separated} \quad \forall \text{pairs } (A(s), B(s)) \tag{5c}$$

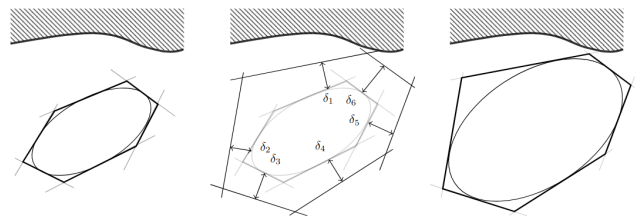


Fig. 5: Process of expanding a convex polytope in c-space

C-IRIS iteratively creates and grows polytopes until they cover a specified fraction of the robot's reachable workspace. In my implementation, I set this parameter to **0.75**.

I used the Drake library [8] for C-IRIS. In Drake uses IPOPT to find the nearest collision boundary from a given configuration when expanding a convex region.

B. Creating the Graph of Convex Sets

After convex decomposition of free-space, the graph is constructed by connecting convex regions that overlap or share an edge. Also the start and goal configurations are connected to the regions that they lie within. The creation of the graph is illustrated by Figure 6 from [1]

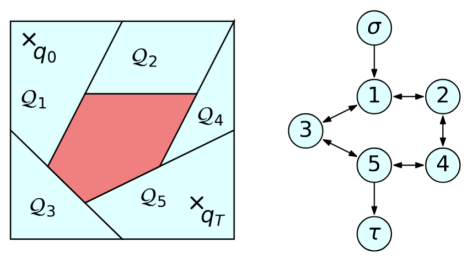


Fig. 6: Constructing the graph of convex sets

C. Solving the MIP

GCS is a MIP because it consists of continuous variables and binary variables. The continuous variables define the path through a convex region. The binary variables determine whether an edge between regions is used or not. The solution of the MIP produces a continuous trajectory through the convex regions, as shown in Figure 7.

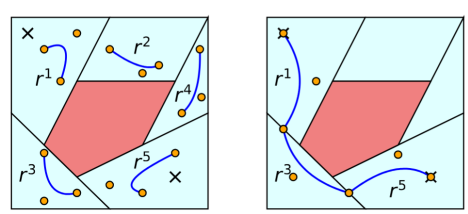


Fig. 7: Trajectory from solving the MIP

$$\min_{r(\cdot),\, h(\cdot)} \quad a\, h(S) + b\, L(r, S) + c\, E\left(\frac{\dot{r}}{\sqrt{\dot{h}(s)}}, S\right) \tag{6a}$$

$$\text{s.t.} \quad r \circ h^{-1} \in \mathcal{C}^{\eta} \tag{6b}$$

$$r(s) \in \bigcup_{i \in \mathcal{I}} \mathcal{Q}_i, \quad \forall s \in [0, S] \tag{6c}$$

$$\dot{r}(s) \in \dot{h}(s)\, \mathcal{D}, \quad \dot{h}(s) > 0, \quad \forall s \in [0, S] \tag{6d}$$

$$h(0) = 0, \quad h(S) \in [T_{\min}, T_{\max}] \tag{6e}$$

$$r(0) = q_0, \quad r(S) = q_T \tag{6f}$$

$$\dot{r}(0) = \dot{h}(0)\dot{q}_0, \quad \dot{r}(S) = \dot{h}(S)\dot{q}_T \tag{6g}$$

Equation 6 is the GCS optimization problem from [1]. It consists of the following components:

- **Objective Function** (6a)**:** Minimize a weighted combination of trajectory time, path length, and energy:
  - $a\, h(S)$: total trajectory time,
  - $b\, L(r, S)$: path length,
  - $c\, E\left(\frac{\dot{r}}{\sqrt{\dot{h}(s)}}, S\right)$: energy.
- **Constraint** (6b)**:** Enforces the trajectory is continuous and can be differentiated η times. Increasing η increases smoothness.
- **Constraint** (6c)**:** Enforces that the trajectory remains within the free, convex regions of c-space.
- **Constraint** (6d)**:** Constrains the velocity.
- **Constraint** (6e)**:** Constrains the duration of the trajectory.
- **Constraint** (6f)**:** Ensures the trajectory starts at the initial configuration q_0 and ends at the goal configuration q_T.
- **Constraint** (6g)**:** Ensures initial and final velocities match what is desired.

I tuned the weights of the objective function (a, b and c), η, and the Bezier order for trajectory segments to produce different path behaviors.

1) Convex Relaxation: The binary variables ϕ_e constrain the problem so only a subset of edges are used. This makes the MIP non-convex and solving the problem can be slow. Therefore, a convex relaxation can be used to speed up the solver. With a convex relaxation, the binary variables become continuous variables in the range of 0 to 1 ($\phi_e \in [0, 1]$). ϕ_e now represents the probability an edge is part of the optimal solution.

GCS rounds over a number of different possible paths to provide a solution. [1] guarantees that $C_{\text{relax}} \leq C_{\text{opt}} \leq C_{\text{round}}$, where C_{relax} is the cost of the best path produced by the convex relaxation, C_{opt} is the cost of the optimal solution found without convex relaxation, and C_{round} is the cost of the path produced by rounding over multiple trajectories.

VI. Results

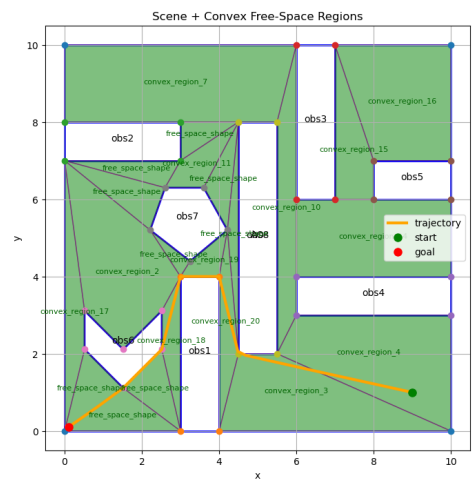
A. Non-smooth Trajectories Produced

To generate non-smooth trajectories I set the parameters to what is shown in Table I. I used convex relaxation with maximum rounded paths set to 5.

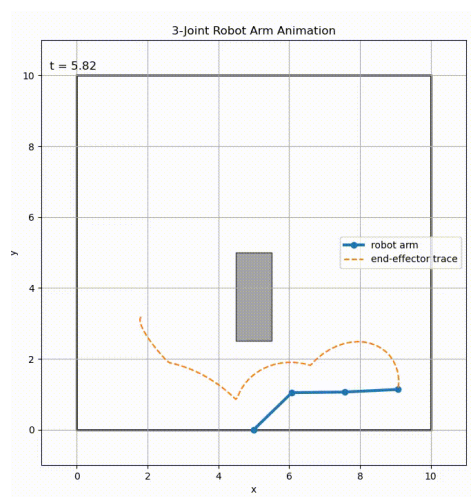
Parameter	Point Robot	Robot Arm
Bézier Order	1	2
Continuity Order	0	0
Path Length Weight	1.0	1.0
Path Energy Weight	0.01	0.001
Time Weight	1.0	1.0

TABLE I: GCS parameter settings to generate non-smooth trajectories.

As shown in Figure 8, the resulting trajectories are jagged and non-smooth.



(a) Point Robot



(b) Robot Arm

Fig. 8: Non-smooth Trajectories

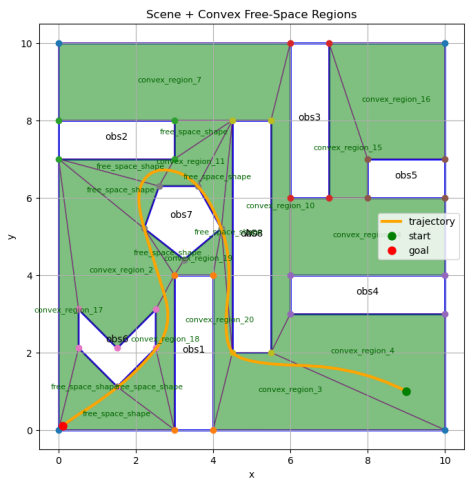
B. Tuning Parameters to Generate Smooth Trajectories

To generate smooth trajectories, I increased the Bezier order(only for the point robot), continuity order η, and energy weight. For the point robot, I decrease the path length and time weights. The parameters I used are shown in Table II

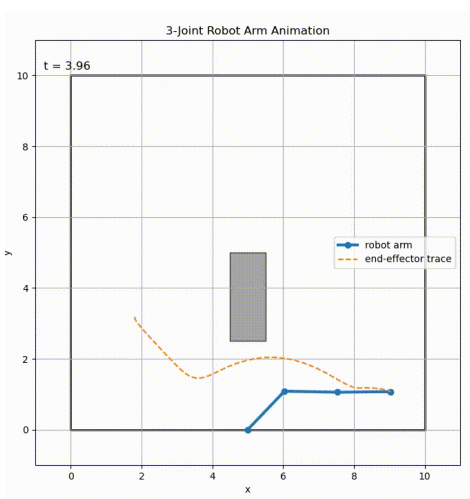
Parameter	Point Robot	Robot Arm
Bézier Order	2	2
Continuity Order	1	1
Path Length Weight	0.1	1.0
Path Energy Weight	100.0	1.0
Time Weight	0.1	1.0

TABLE II: GCS parameter settings to generate smooth trajectories.

The smooth trajectories produced are shown in Figure 9.



(a) Point Robot



(b) Robot Arm

Fig. 9: Smooth Trajectories

C. Testing the tightness of the convex relaxation

To assess the tightness of the convex relaxation and the effectiveness of the rounding procedure, I ran GCS on both the point robot and robot arm problems using the original mixed-integer formulation and its convex relaxation. For these experiments, I set wach weight in the objective function to 1.0.

As you can see from Tables III and IV, the guarantee that $C_{\text{relax}} \leq C_{\text{opt}} \leq C_{\text{round}}$ held. Therefore, the rounding procedure effectively converts the convex relaxation solution into a feasible trajectory that is optimal or near-optimal.

Method	Num. Rounded Paths	Solver	Cost
C_{relax}	0	MOSEK	65.3318
C_{round}	4	MOSEK	67.1045
C_{opt}	0	Gurobi	67.1045

TABLE III: Comparison of C_{relax}, C_{round}, and C_{opt} for the point robot problem

Method	Num. Rounded Paths	Solver	Cost
C_{relax}	0	MOSEK	21.4204
C_{round}	5	MOSEK	21.6025
C_{opt}	0	Gurobi	21.6025

TABLE IV: Comparison of C_{relax}, C_{round}, and C_{opt} for the robot arm problem

Additionally, I examined the relaxed binary variables ϕ_e returned by the convex relaxation with no rounding. The histograms in Figure 10 show that many edges have ϕ_e values between 0 and 1. With no rounding, the trajectory produced by convex relaxation is dispersed over many edges and is not feasible for the actual robot. Figure 11 shows the infeasible trajectory that was returned by convex relaxation without rounding for the point robot problem.

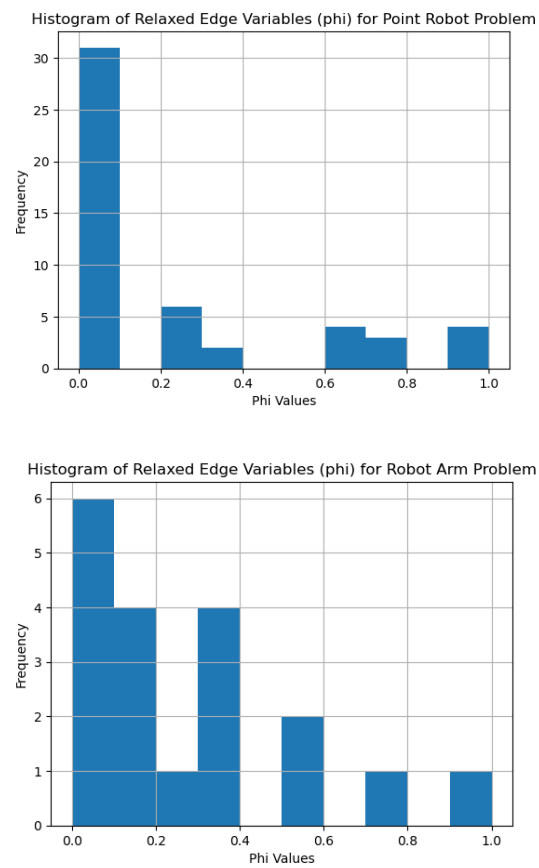


Fig. 10: Histograms of relaxed binary variables ϕ_e from convex relaxation

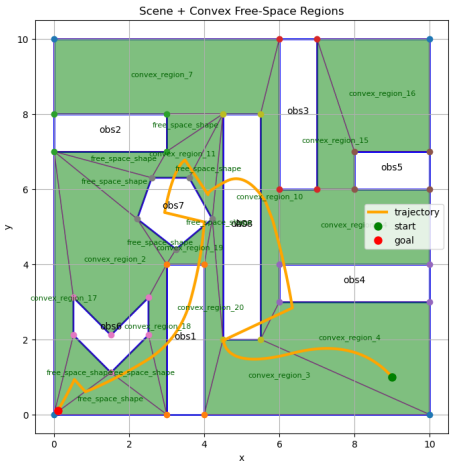


Fig. 11: Infeasible trajectory produced by convex relaxation with no rounding

D. Assessing Computational Efficiency

For the robot arm problem, convex relaxation with rounding took 0.0286s while the traditional MIP (no relaxation) took 0.3584, making convex relaxation around 12.5 times faster.

C-IRIS was very expensive, taking 10.25s to complete on average. It took around 98.7% of total runtime on average.

VII. Conclusion

GCS effectively generates trajectories for robots that satisfy explicitly defined convex constraints. With the help of C-IRIS convex decomposition, it can generate trajectories in high-dimensional, complex configuration spaces.

Additionally, the convex relaxation for GCS was not tight, with many edges having ϕ_e values between 0 and 1. However, the rounding step helps convert the solution produced by the convex relaxation into a feasible, near-optimal solution. Convex relaxation with rounding was much faster than the traditional MIP.

When running the code, C-IRIS was very slow. Therefore, C-IRIS + GCS is not suited for a dynamic environment as it would require C-IRIS to be executed every time an obstacle changes its position. It is best suited for a static environment where C-IRIS only needs to be executed once.

Future work includes extending this framework to incorporate Riemannian manifolds into the optimization problem. This would enable GCS to handle angle wrap-around so that it can generate trajectories for robots with continuous joints.

VIII. Use of Gen AI

Please note that I used ChatGPT to help me with Drake syntax in my code. I also used it to help me generate videos and images for the report and presentation. This amplified my learning because it helped me fully understand the capabilities of the Drake library.

References

[1] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," 2022. [Online]. Available: https://arxiv.org/abs/2205.04422

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[3] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," 2011. [Online]. Available: https://arxiv.org/abs/1105.1186

[4] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "Chomp: Gradient optimization techniques for efficient motion planning," in *2009 IEEE International Conference on Robotics and Automation*, 2009, pp. 489–494.

[5] The CGAL Project, "CGAL: Computational geometry algorithms library," https://www.cgal.org/, 2026, accessed: April 22, 2026.

[6] N. Atanasov, "Ece276b: Planning & learning in robotics – lecture 5: Configuration space," https://natanaso.github.io/ece276b/, 2023, university of California San Diego, Lecture Notes.

[7] H. Dai, A. Amice, P. Werner, A. Zhang, and R. Tedrake, "Certified polyhedral decompositions of collision-free configuration space," 2023. [Online]. Available: https://arxiv.org/abs/2302.12219

[8] Drake Development Team, "Drake: Model-based design and verification for robotics," https://drake.mit.edu/, accessed: 2026-04-22.

[9] T. Cohn, M. Petersen, M. Simchowitz, and R. Tedrake, "Non-euclidean motion planning with graphs of geodesically-convex sets," 2023. [Online]. Available: https://arxiv.org/abs/2305.06341

[10] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM Journal on Optimization*, vol. 34, no. 1, p. 507–532, Feb. 2024. [Online]. Available: http://dx.doi.org/10.1137/22M1523790

[11] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Workshop on the Algorithmic Foundations of Robotics*, 2014. [Online]. Available: https://api.semanticscholar.org/CorpusID:9135142